

# WEB TECHNOLOGIES

## CSE-4004



### *LECTURE 7: Event Handling*



**By: Dr.MD.Sirajuddin**

## **EVENT**

- It is an action that occurs in the browser.
- Triggered by user interaction or by the browser itself.
- Allows webpage to respond to the user and browser actions.

### **User Interaction Events**

- Mouse events, Keyboard events, Form events.

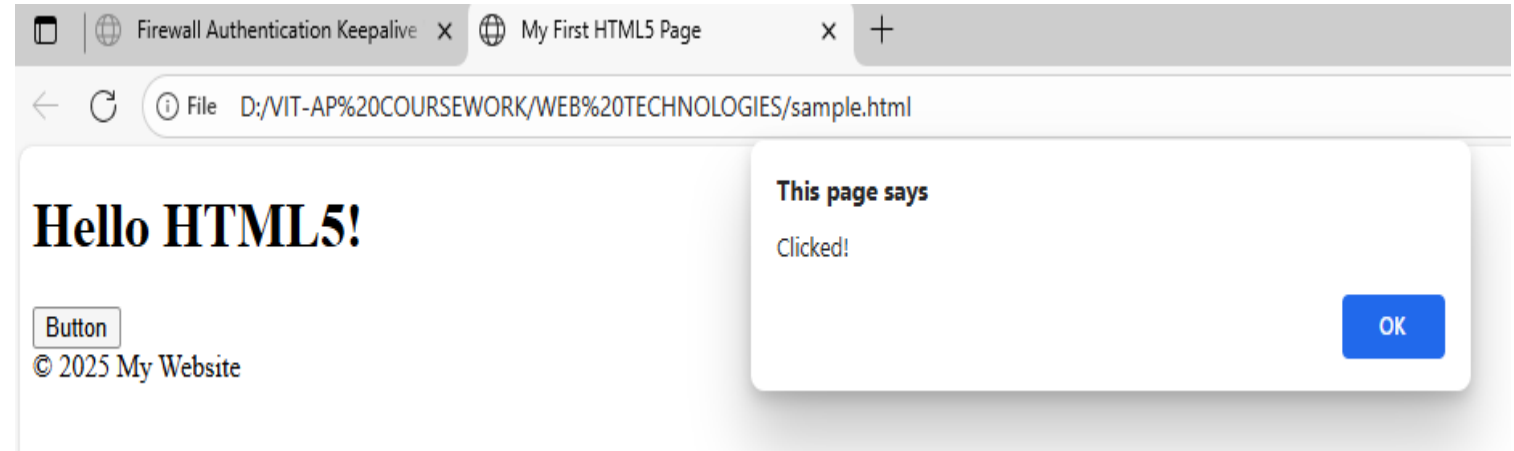
### **Browser Events**

- onload, onresize, onunload etc.
- `<body onload="alert('page loaded')">`

# MOUSE EVENTS

- **onclick**
- **ondblclick**
- **onmousedown**
- **onmouseup**
- **onmousemove**
- **onmouseover**
- **onmouseout**
- **onwheel**

**<button  
onclick="alert('Clicked!')">Button</button>**



## KEYBOARD EVENTS

- onkeyup
- onkeydown: fires for all keys
- Onkeypress: fires only for printable characters

## FORM EVENTS

- onfocus
- onblur
- onchange
- onsubmit
- onreset

## WINDOW EVENTS

- onload
- onunload
- onresize
- onscroll

## **Event Handling**

- **It is important because it allows web pages to react instantly to user actions, make the interface dynamic and interactive.**

## **Event Handler**

- **It is a JS function that is executed in response to a specific event.**

| <b>Event Handler</b>                                                                                          | <b>Event Listeners</b>                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assigned directly to an element. It is a JavaScript function that is executed in response to an event.</b> | <b>This is the mechanism that "listens" for a specific type of event to occur on a particular element</b>                                                                                                |
| <b>Combines HTML with JS (Inline)</b>                                                                         | <b>Keeps HTML and JS separate</b>                                                                                                                                                                        |
| <b>Only one handler per event type. A new assignment overwrites any previous one.</b>                         | <b>Multiple listeners for same event</b>                                                                                                                                                                 |
| <b>&lt;button<br/>onclick="alert('Hello!')"&gt;Click<br/>Me&lt;/button&gt;</b>                                | <b>&lt;button id="but1"&gt;Click Me&lt;/button&gt;<br/>const btn =<br/>document.getElementById('but1');<br/>btn.addEventListener('click', function()<br/>{<br/>    alert('Button clicked!');<br/>});</b> |

# Event Propagation

➤ Event propagation in JavaScript is the process that defines how events travel through the DOM tree, determining the order in which elements receive the event.

➤ There are three phases

## 1. Event Capturing Phase:

➤ The event starts from the root of the DOM and travels down to the target element.

➤ If you set an event listener with `{ capture: true }`, it will trigger during this phase.

➤ **Example:** Clicking a button inside a `<div>` will first check if the `<div>` has a capturing listener before reaching the button.

## 2. Target Phase

- The target phase is the moment when the event has reached the actual element that was interacted with (the event's target).
- The target phase is always executed — you can't skip it.

## 3. Event Bubbling Phase

- After reaching the target element, the event bubbles up through its ancestors.
- By default, most event listeners work in the bubbling phase.
- Example: Clicking a button inside a `<div>` will trigger the button's handler first, then the `<div>`'s handler, and so on up to `<body>` and `<html>`.



```
element.addEventListener(event, function, useCapture);
```

**element:** It is the DOM element you want to attach the event to

**event:** A string representing the event type.

**function:** It is a function to execute when the event occurs.  
Can be an anonymous function or a named function.

*Example: `function() { alert('Clicked!'); } or myFunction`*

**useCapture** (optional): Boolean type to specify event phase.  
true → event capturing phase  
false → event bubbling phase (default)

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Capture Phase  
  Demo</title>  
</head>  
<body>  
  <div id="outer">Outer DIV  
    <button id="inner">  
      Click Me  
    </button>  
  </div>
```

```
<script>  
const outer =  
document.getElementById('outer');  
const inner =  
document.getElementById('inner');  
document.addEventListener('click', () =>  
{  
  alert('Document listener (capture)');  
}, true);  
  
outer.addEventListener('click', () => {  
  alert('Outer DIV listener (capture)');  
}, true);
```

```
inner.addEventListener('click', () => {  
  alert('Button listener (capture)');  
}, true);  
</script>  
</body>  
</html>
```

[Example](#)

## **event.stopPropagation():**

- **Stops the event from bubbling up (or capturing down) further in the DOM.**

## **event.preventDefault():**

- **Prevents the default browser action (event bubbling).**

## Event Delegation

- It is a technique in which a single event listener is attached to a parent element to handle events triggered by its child elements, using the concept of event bubbling.

```
<div id="parent">  
  <button>Save</button>  
  <button>Delete</button>  
</div>
```

```
<script>
```

```
document.querySelector("#parent").addEventListener("click", event =>  
{  
  console.log(`You clicked: ${event.target.textContent}`);  
} );
```

```
</script>
```

**DOMContentLoaded:** is an event that occurs when the browser has completely built the DOM tree.

**Syntax:1**

```
document.addEventListener("DOMContentLoaded", function () {  
    // code to run after DOM is loaded  
});
```

**Syntax:2**

```
document.addEventListener("DOMContentLoaded", () => {  
    // code to run after DOM is loaded  
});
```

**Example:**

```
document.addEventListener("DOMContentLoaded", () => {  
    document.body.innerText = "Hello";  
});
```

```
document.addEventListener("DOMContentLoaded", showMessage);  
  
function showMessage() //executed when DOM is ready  
{  
    document.body.innerText = "DOM is ready";  
}
```

```
<!DOCTYPE html>
<html>
<body>
  <h1><p id="pid">Hai</p></h1>
  <script>
    const p = document.getElementById("pid");
    setTimeout(() => {
      p.innerText = "Hello";
    }, 2000); //after 2sec the text inside p change to "Hello"
  </script>
</body>
</html>
```

Example